

Измерение и тестирование пропускной способности сети. Воспроизводимый эксперимент

Лабораторная работа № 3

Шулуужук Айраана НПИбд-02-22

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Создание простейшей топологии сети	6
2.2	Построение графиков по проводимому эксперименту	11
3	Выводы	19

Список иллюстраций

2.1	создание необходимых каталогов	6
2.2	скрипт	7
2.3	создание топологии сети	8
2.4	внесения изменений в скрипт для вывода информации о хостах	9
2.5	топология сети	9
2.6	внесение изменений в скрипт	10
2.7	lab_iperf3_topo2.py	11
2.8	lab_iperf3_topo.py	11
2.9	добавление библиотек и изменение параметров на хостах .	12
2.10	внесение изменений в скрипт	12
2.11	топология сети	13
2.12	создание графиков и Makefile	13
2.13	Makefile	13
2.14	проверка отработки Makefile	14
2.15	полученные графики	14
2.16	количество переданных файлов	15
2.17	окно перегрузки	15
2.18	максимальная единица передачи	16
2.19	повторная передача	16
2.20	время приема-передачи	17
2.21	отклонение времени приема-передачи	17
2.22	пропускная способность	18

Список таблиц

1 Цель работы

Основной целью работы является знакомство с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получение навыков проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.

2 Выполнение лабораторной работы

2.1 Создание простейшей топологии сети

В каталоге `/work/lab_iperf3` для работы над проектом создадим каталог `lab_iperf3_topo` и скопируем в него файл с примером скрипта `mininet/examples/emphynet.py`, описывающего стандартную простую топологию сети mininet (рис. 2.1)

```
mininet@mininet-vm:~/work/lab_iperf3$ ls
iperf.csv  iperf_result.json  lab_iperf3_topo  results
mininet@mininet-vm:~/work/lab_iperf3$ cd ~/work/lab_iperf3/lab_iperf3_topo
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cp ~/mininet/examples/emphynet.py
~/work/lab_iperf3/lab_iperf3_topo
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ ls
emphynet.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mv emphynet.py lab_iperf3_topo.py
```

Рис. 2.1: создание необходимых каталогов

Изучим содержание скрипта `lab_iperf3_topo.py` (рис. 2.2)

```

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

def emptyNet():
    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n' )
    net.start()

    info( '*** Running CLI\n' )
    CLI( net )

    info( '*** Stopping network\n' )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
mininet@mininet-vm:~/work/lab_inerf3/lab_inerf3_tono$

```

Рис. 2.2: скрипт

Запустим скрипт создания топологии и после отработки скрипта посмотрите элементы топологии (рис. 2.3)

```

if __name__ == '__main__':
    setLogLevel('info')
    emptyNet()
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
*** Running CLI
*** Starting CLI:
mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
s3 lo: s3-eth1:h1-eth0 s3-eth2:h2-eth0
c0
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=928>
<Host h2: h2-eth0:10.0.0.2 pid=932>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=937>
<Controller c0: 127.0.0.1:6653 pid=921>
mininet> ex

```

Рис. 2.3: создание топологии сети

Внесем в скрипт `lab_iperf3_topo.py` изменение, позволяющее вывести на экран информацию для двух хостов, а именно имя хоста, IP-адрес, MAC-адрес (рис. 2.4)


```

/home/mi~ topo.py [----] 8 L:[ 11+25 36/ 48] *(825 /1071b) 116 0x074 [*][X]
from mininet.log import setLogLevel, info

def emptyNet():
    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( "*** Adding controller\n" )
    net.addController( 'c0' )

    info( "*** Adding hosts\n" )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( "*** Adding switch\n" )
    s3 = net.addSwitch( 's3' )

    info( "*** Creating links\n" )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( "*** Starting network\n" )
    net.start()

    print( "Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC() )

    info( "*** Running CLI\n" )
    CLI( net )

    info( "*** Stopping network\n" )
    net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()

```

Рис. 2.4: внесения изменений в скрипт для вывода информации о хостах

Проверим корректность выполнения вывода (рис. 2.5)

```

mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ... I
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 16:63:43:07:1a:d9
*** Running CLI
*** Starting CLI:
mininet>

```

Рис. 2.5: топология сети

Mininet предоставляет функции ограничения производительности и изоляции с помощью классов `CPULimitedHost` и `TCLink`. - Добавим в

скрипт настройки параметров производительности. В начале скрипта `lab_iperf3_topo2.py` добавим записи об импорте классов `CPULimitedHost` и `TCLink`. - Изменим строку описания сети, указав на использование ограничения производительности и изоляции - Изменим функцию задания параметров виртуального хоста `h1`, указав, что ему будет выделено 50% от общих ресурсов процессора системы, для хоста 2 выделим 45% - Изменим функцию параметров соединения между хостом `h1` и коммутатором `s3` (рис. 2.6)

```
/home/mi~topo2.py [-M--] 78 L:[ 1+31 32/ 52] *(915 /1336b) 32 0x020 [*][X]
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.node import CPUimitedHost
from mininet.link import TCLink

def emptyNet():
    """Create an empty network and add nodes to it."""

    net = Mininet( controller=Controller, waitConnected=True, host = CPUimitedHost

    info( '*** Adding controller\n' )
    net.addController( 'c3' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1', cpu=50 )
    h2 = net.addHost( 'h2', ip='10.0.0.2', cpu=45 )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3, bw=10, delay='5ms', max_queue_size=1000, loss=10, useE
    net.addLink( h2, s3 )

    info( '*** Starting network\n' )
    net.start()

    print( 'Host', h1.name, 'has IP address', h1.IP(), 'and MAC address', h1.MAC
```

Рис. 2.6: внесение изменений в скрипт

Запустим на отработку сначала скрипт `lab_iperf3_topo2.py`, затем `lab_iperf3_topo.py` и сравните результат (рис. 2.7) (рис. 2.8)

```

mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo2.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(10.00Mbit 5ms delay 10.00000% loss) (10.00Mbit 5ms delay 10.00000% loss) *** Starting network
*** Configuring hosts
h1 (cfs 5000000/1000000us) h2 (cfs 4500000/1000000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (10.00Mbit 5ms delay 10.00000% loss) ... (10.00Mbit 5ms delay 10.00000% loss)
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 36:bd:d6:74:51:f7
Host h2 has IP address 10.0.0.2 and MAC address 4e:25:b8:ae:64:ca
*** Running CLI
*** Starting CLI:
mininet>

```

Рис. 2.7: lab_iperf3_topo2.py

```

mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ... I
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address d2:b4:52:61:69:0e
Host h2 has IP address 10.0.0.2 and MAC address 46:57:42:f4:91:87
*** Running CLI
*** Starting CLI:
mininet>

```

Рис. 2.8: lab_iperf3_topo.py

2.2 Построение графиков по проводимому эксперименту

Сделаем копию скрипта lab_iperf3_topo2.py и поместите его в подкаталог iperf. В начале скрипта lab_iperf3.py импортируем библиотеку time. Изменим код в скрипте lab_iperf3.py так, чтобы: – на хостах не было ограничения по использованию ресурсов процессора; – каналы между хостами и коммутатором были по 100 Мбит/с с задержкой 75 мс, без потерь, без использования ограничителей пропускной способности и максимального

размера очереди (рис. 2.9).

```
/home/mi~perf3.py  [-M--] 46 L: [ 1+32 33/ 52] *(926 /1320b) 41 0x029 [*][X]
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

import time
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.node import CPULimitedHost
from mininet.link import TCLink

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True, host = CPULimitedHost

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip= '10.0.0.1' )
    h2 = net.addHost( 'h2', ip= '10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3, bw=100, delay='75ms' )
    net.addLink( h2, s3, bw=100, delay='75ms' )

    info( '*** Starting network\n' )
    net.start()
```

Рис. 2.9: добавление библиотек и изменение параметров на хостах

После функции старта сети опишем запуск на хосте h2 сервера iPerf3, а на хосте h1 запуск с задержкой в 10 секунд клиента iPerf3 с экспортом результатов в JSON-файл, прокомментируем строки, отвечающие за запуск CLI-интерфейса: (рис. 2.10)

```
info( '*** Starting network\n' )
h2.cmdPrint( 'iperf3 -s -B -I' )
time.sleep(10)
h1.cmdPrint( 'iperf3 -c', h2.IP(), '-x2 -j iperf_result.json' )

print( "Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC
print( "Host", h2.name, "has IP address", h2.IP(), "and MAC address", h2.MAC

#info( '*** Running CLI\n' )
#CLI( net )

info( '*** Stopping network\n' )
net.stop()

if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

Рис. 2.10: внесение изменений в скрипт

Запустим итоговый скрипт (рис. 2.11)

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ sudo python lab_iperf3.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00M
bit 75ms delay) *** Starting network
*** Configuring hosts
h1 (cfs -1/1000000us) h2 (cfs -1/1000000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) ... (100.00Mbit 75ms delay) (1
00.00Mbit 75ms delay)
*** Waiting for switches to connect
s3
*** Starting network
*** h2 : ('iperf3 -s -D -1',)
*** h1 : ('iperf3 -c', '10.0.0.2', '-J > iperf_result.json')
Host h1 has IP address 10.0.0.1 and MAC address d6:09:9b:02:95:c6
Host h2 has IP address 10.0.0.2 and MAC address 26:c8:e2:16:91:76
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2
*** Done
```

Рис. 2.11: топология сети

Построим графики из получившегося JSON-файла (рис. 2.12)

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ touch Makefile
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls
iperf.csv iperf_result.json lab_iperf3.py Makefile results
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 2.12: создание графиков и Makefile

Создадим Makefile для проведения всего эксперимента. В Makefile пропишем запуск скрипта эксперимента, построение графиков и очистку каталога от результатов (рис. 2.13)

```
/home/mi~Makefile [-M--] 23 L:[ 1+10 11/ 11] *(178 / 178b) <EOF>
all: iperf_result.json plot

iperf_result.json:
----->sudo python lab_iperf3.py

plot: iperf_result.json
----->plot_iperf.sh iperf_result.json

clean:
----->-rm -f *.json *.csv
----->-rm -rf results
```

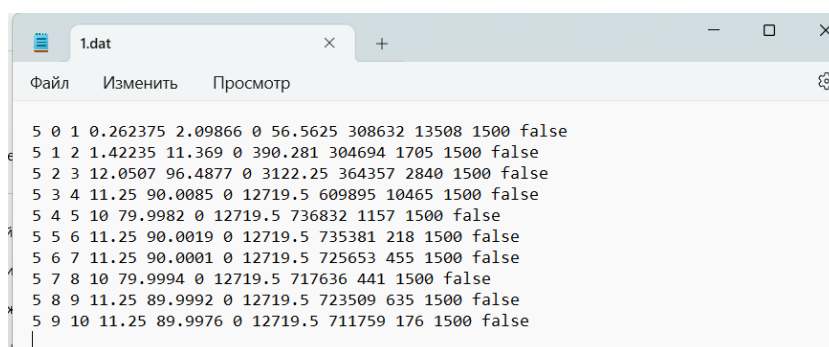
Рис. 2.13: Makefile

Проверим корректность отработки Makefile (рис. 2.14) (рис. 2.15)

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ make clean
rm -f *.json *.csv
rm -rf results
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ make
sudo python lab_iperf3.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) (100.00M
bit 75ms delay) *** Starting network
*** Configuring hosts
h1 (cfs -1/100000us) h2 (cfs -1/100000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (100.00Mbit 75ms delay) (100.00Mbit 75ms delay) ... (100.00Mbit 75ms delay) (1
00.00Mbit 75ms delay)
*** Waiting for switches to connect
s3
*** Starting network
*** h2 : ('iperf3 -s -D -1',)
*** h1 : ('iperf3 -c', '10.0.0.2', '-J > iperf_result.json')
Host h1 has IP address 10.0.0.1 and MAC address a6:9c:58:39:3c:8a
Host h2 has IP address 10.0.0.2 and MAC address 8a:e2:a0:41:ac:34
*** Stopping network*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2
*** Done
plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 2.14: проверка отработки Makefile

Полученные графики (рис. 2.15) (рис. 2.16) (рис. 2.17) (рис. 2.18) (рис. 2.19) (рис. 2.20) (рис. 2.21) (рис. 2.22)



```
1.dat
Файл  Изменить  Просмотр

5 0 1 0.262375 2.09866 0 56.5625 308632 13508 1500 false
5 1 2 1.42235 11.369 0 390.281 304694 1705 1500 false
5 2 3 12.0507 96.4877 0 3122.25 364357 2840 1500 false
5 3 4 11.25 90.0085 0 12719.5 609895 10465 1500 false
5 4 5 10 79.9982 0 12719.5 736832 1157 1500 false
5 5 6 11.25 90.0019 0 12719.5 735381 218 1500 false
5 6 7 11.25 90.0001 0 12719.5 725653 455 1500 false
5 7 8 10 79.9994 0 12719.5 717636 441 1500 false
5 8 9 11.25 89.9992 0 12719.5 723509 635 1500 false
5 9 10 11.25 89.9976 0 12719.5 711759 176 1500 false
```

Рис. 2.15: полученные графики

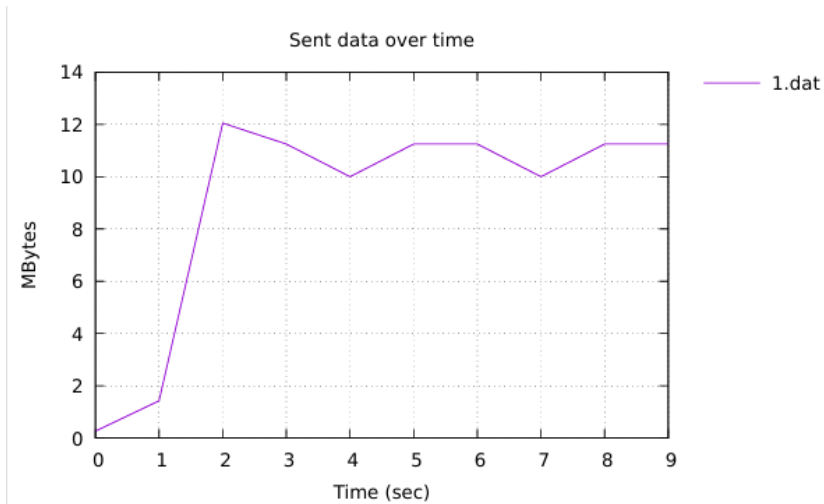


Рис. 2.16: количество переданных файлов

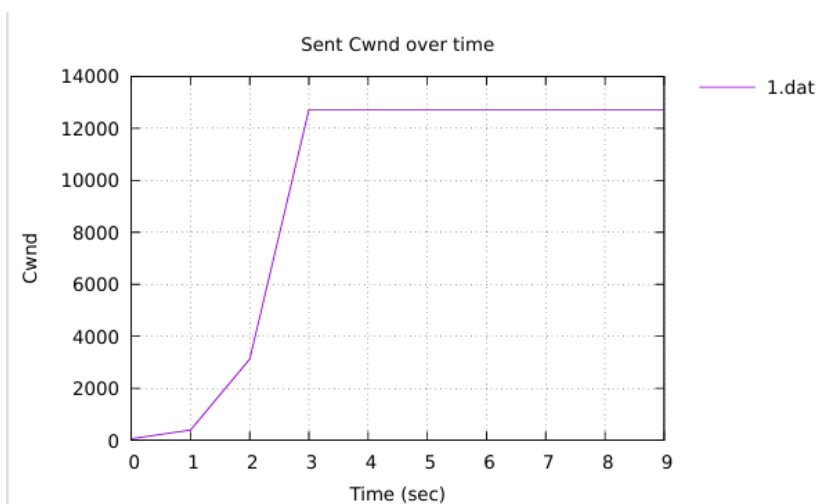


Рис. 2.17: окно перегрузки

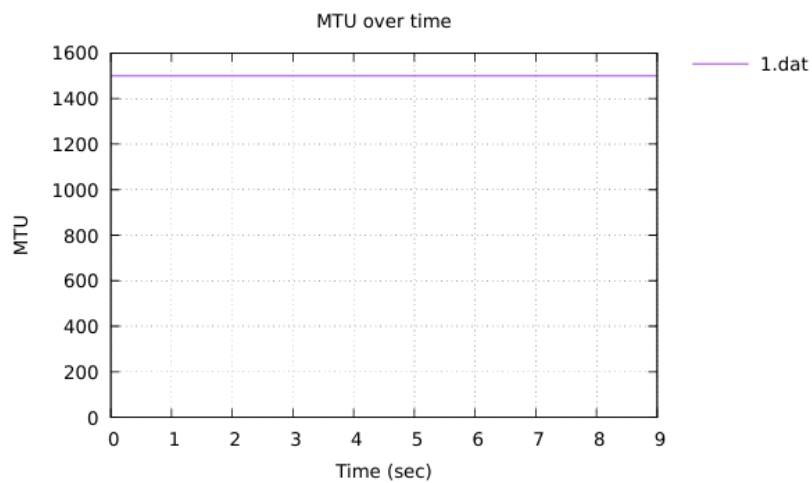


Рис. 2.18: максимальная единица передачи

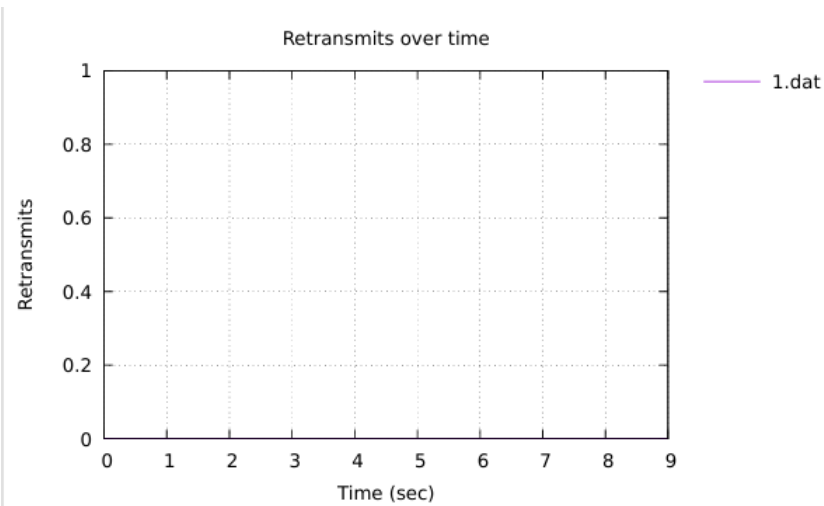


Рис. 2.19: повторная передача

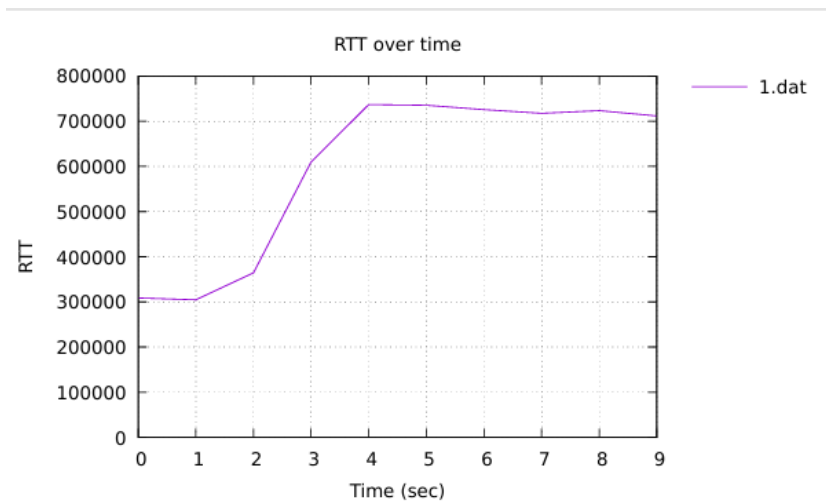


Рис. 2.20: время приема-передачи

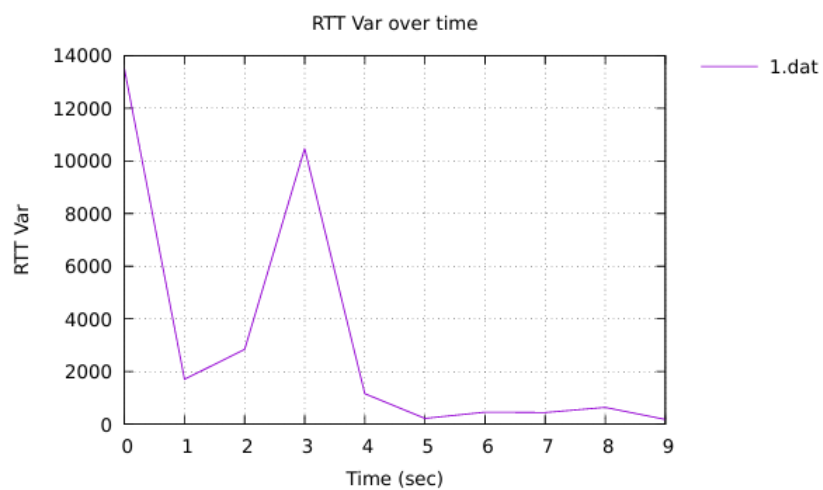


Рис. 2.21: отклонение времени приема-передачи

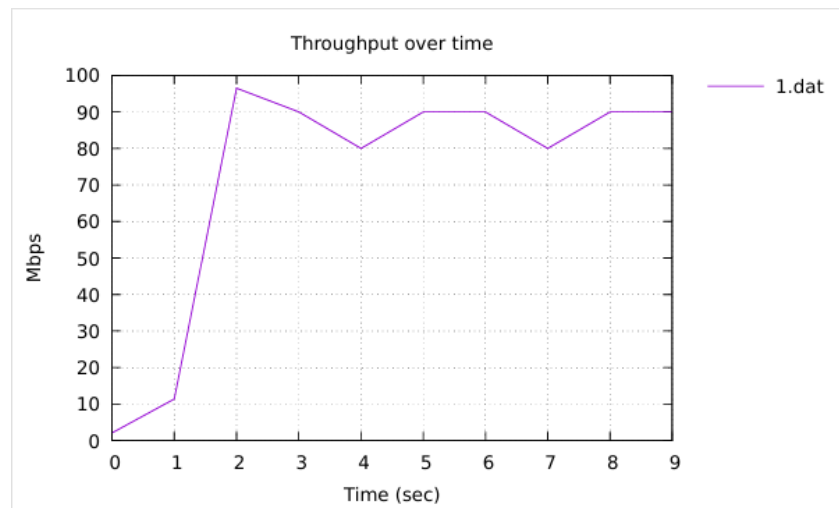


Рис. 2.22: пропускная способность

3 Выводы

В результате выполнения лабораторной работы было проведено знакомство с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получение навыков проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.